

ngspice-46 — Full Change Report

Every modification and its reason (original → current)

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice-46 — Full Change Report	3
1. The OSDI ABI, version 0.4 → 0.7	3
2. The OSDI runtime — src/osdi/	4
3. Analyses — src/spicelib/analysis/	5
4. Frontend — src/frontend/	7
5. Parser and device registry — src/spicelib/	8
6. Maths — src/math/	8
7. Build system	10
8. Per-enhancement index	10

ngspice-46 — Full Change Report

Every modification applied to ngspice-46 in this project, with its reason. The baseline is the pristine ngspice-46 source tree (as vendored in the project's `original/` snapshot, which already contains OpenVAF-Reloaded's stock OSDI support); the current state is the tree committed in this repository under `ngspice-46/`. Each entry links the enhancement write-up in [enhancements_doc/](#) that carries the full engineering detail and the verifying example suite. The companion document [openvaf_changes_full-report.md](#) covers the compiler side.

Maintenance note: this report is updated whenever an enhancement touches ngspice sources. If you are reading it alongside a newer tree, the per-enhancement index at the end tells you the last change it covers.

Scope summary. One new source file and 41 modified ones carry all the functional changes (~2,000 substantive diff lines). A further ~100 `Makefile.in` files and `config.h.in` differ only because the auto-tools were regenerated with a current libtool ([E-77](#)); they contain no hand-written changes and are not listed individually. Everything below is behavior, interface, or diagnostics.

1. The OSDI ABI, version 0.4 → 0.7

`src/osdi/osdi.h` is the authoritative C contract between ngspice and the `.osdi` objects `openvaf-r` produces. The project extended it four times — twice additively (new exported symbols old loaders simply never look up), twice with genuine version bumps (layout changes):

Change	Kind	Enhancement	Why
<code>OSDI_ABSDELAY_COUNT</code> / <code>OSDI_ABSDELAY_INFO</code> exports	additive sym- bols	E-1	<code>absdelay()</code> needs simulator-side waveform history and synthetic delay-equation rows; the descriptors tell ngspice which nodes/offsets each delay slot uses
<code>OSDI_LAST_CROSSING_COUNT</code> / <code>OSDI_LAST_CROSSING_INFO</code> exports	additive sym- bols	E-6	<code>last_crossing()</code> needs waveform history and crossing interpolation — same additive pattern as <code>absdelay</code>
<code>OsdiNode.nodeset</code> field	ABI 0.5 (node stride change)	E-45	Verilog-A net initializers (<code>electrical a = 5.0;</code>) become solver initial-guess nodesets; every node record carries the hint
<code>num_ac_stim_src</code> / <code>ac_stim_sources</code> / <code>load_ac_stim</code> descriptor tail	ABI 0.6 (de- scriptor grows)	E-51	full <code>ac_stim()</code> support: a Verilog-A module can <i>be</i> the AC stimulus, so the descriptor must enumerate its AC right-hand-side sources
<code>load_noise</code> destination stride 1 → 2 (signed (<code>flat</code> , <code>react</code>) power pairs)	ABI 0.7	E-54	correct noise physics: operating-point-dependent factors and <code>ddt()</code> -shaped ($j\omega$) noise need a complex amplitude per source, $re + j\omega \cdot im$, so coherent same-named sources (LRM 4.6.4) sum with correct sign and frequency shape

Two additive flag conventions ride on the existing `eval()` interface (not version bumps):

- `EVAL_FLAG_IS_FINAL_STEP` (bit 1 $\ll$ 21) in the `eval input` flags — `@(final_step)` firing ([E-53](#));
- `eval return` flags for `$finish/$stop` requests and `$discontinuity`-driven step rejection ([E-55](#)).

Pairing consequence: this ngspice requires $\text{OSDI} \geq 0.7$ and rejects older objects with a clear version message; `.osdi` files from older compilers must be recompiled ([E-54](#)).

2. The OSDI runtime — `src/osdi/`

osdiaccept.c — new file ([E-55](#), [E-1](#), [E-6](#))

The accepted-timepoint hook (`DEVaccept`) OSDI previously lacked. It advances the `absdelay/last_crossing` waveform histories only at *accepted* points (so rejected timesteps don't corrupt the delay lines) and reads the per-attempt latched `$finish/$stop` request flags at the one boundary where honoring them is safe. Why: acting on `$stop` mid-Newton-iteration was treated as a step failure and ground the timestep into a rejection loop; `$finish` was ignored outright.

osdidefs.h (104 diff lines)

- `OsdiExtraInstData` grows the per-instance fields behind every runtime feature: `dt/temp` offsets with given-flags (instance temperature), `eval_flags`, `absdelay` waveform-history rows and pre-allocated KLU/SPARSE matrix-pointer sets for the delay-equation rows (re-pointed on every DC↔AC transition, mirroring the regular Jacobian handling), and the `last_crossing` history ([E-1](#), [E-6](#), [E-55](#)).
- `ALIGN` macro renamed `OSDI_ALIGN`: the macOS SDK's `<sys/param.h>` defines an unrelated `ALIGN`, producing a redefinition warning in every OSDI translation unit ([E-77](#)).

osdiitf.h / **osdiext.h**

The registry-entry structure gains the `absdelay/last_crossing` descriptor pointers, and `OSDIpendingRequests()` is exported so the analyses can ask "did any device request `$finish/$stop` at this accepted point?" ([E-1](#), [E-6](#), [E-55](#)).

osdiregistry.c (68 diff lines)

- Loads the `absdelay/last_crossing` descriptor arrays from each `.osdi` and threads them into the registry entries ([E-1](#), [E-6](#)).
- Requires $\text{OSDI} \geq 0.7$ ([E-54](#)).

osdiinit.c (8 diff lines)

- Registers the `DEVaccept` hook ([E-55](#)).
- Publishes the synthetic instance-temperature parameter under both spellings, `dt` and the conventional `dtemp` every built-in device uses; `n1 a b mod dtemp=10` used to fail with "unknown parameter" while the underlying plumbing was exact ([E-80](#)).

osdiload.c (441 diff lines — the largest OSDI change)

- Stamps the `absdelay` synthetic rows (`y_synth`, `z`) for DC, AC and transient, driving the delay from the recorded history ([E-1](#)).
- Evaluates `last_crossing` from the recorded waveform with linear interpolation between the bracketing timepoints ([E-6](#)).
- Passes the final-step flag through to `eval()` ([E-53](#)).
- Latches `eval`-return flags per timepoint attempt (`point_eval_flags`, OR-ed across Newton iterations, reset per attempt) so `$finish/$stop` under solution-dependent conditions survive to the accepted-point boundary ([E-55](#)).

- Folds the stride-2 signed noise-power pairs ($\text{fac} \cdot |\text{fac}|$ semantics) when transferring `load_noise` results (E-54, E-42).

osdisetup.c (178 diff lines)

- Creates the synthetic delay-equation unknowns and matrix entries for `absdelay` (E-1).
- Applies the OSDI nodesets (`OsdiNode.nodeset`) as solver initial guesses (E-45).
- A `$fatal/$finish` raised during setup (models rejecting their configuration by design — HiSIM's `$port_connected` guards) used to surface as ngspice's baffling *"impossible error — can't occur"*; it now reads *"a Verilog-A device rejected its configuration during setup"* (E-56).
- An internal OSDI node that appears in no Jacobian entry — a net structurally decoupled from the matrix, e.g. an explicit ground `gnd` reference whose branch contribution `V(p,gnd) <+ ...` drops the `gnd` column — used to be allocated its own solver row. That all-zero row/column is benign under Sparse (it decouples to $V=0$) but makes the KLU matrix structurally singular, so KLU returned a wrong DC answer (groundcontrib: $v(p)=0$ not 1.5; hierbranch: branch currents 0). Such nodes are now tied to ground (node 0) at setup, matching how OpenVAF already treats them and fixing both solvers (E-116).

osdiacld.c (89 diff lines)

- AC loading gains the `ac_stim` right-hand-side injection: the partitioned AC-stimulus source array is loaded through `load_ac_stim` and added to the AC RHS with exact magnitude and phase, `$mfactor` applied linearly (deterministic signals scale with multiplicity) (E-51, E-26).
- Complex ($j\omega$ -shaped) noise coupling entries participate in the AC matrix (E-54).

osdinoise.c (93 diff lines)

- Per-instance grouping of same-named noise sources: coherent summation of complex amplitudes $(a + j\omega \cdot b) \cdot T$ before squaring, per LRM 4.6.4 — same-named sources correlate (including exact anti-phase cancellation), differently-named ones stay independent (E-42).
- Reads the stride-2 signed pairs of ABI 0.7 (E-54).

osditrunc.c (34 diff lines)

- Honors `$discontinuity(n)`'s next-step clamp: a negative `bound_step` sentinel from the model limits the step after the event (E-24).
- Honors the step-rejection return flag: when an event fired inside the step, `OSDitrunc` requests `delta/8` (with a $20 \cdot \text{CKTdelmin}$ termination floor) so the integrator bisects onto the discontinuity instead of extrapolating across it (E-55).

osdi/Makefile.am

Adds `osdiaccept.c` to the build (E-55).

3. Analyses — **src/spicelib/analysis/**

dctran.c (46 diff lines)

- Advances OSDI delay/crossing histories via the accept path and skips history corruption on rejected steps (E-1, E-6).
- Issues the dedicated `@(final_step)` evaluation at the converged end of a successful transient (E-53).

- Honors accepted-point `$finish` (ends the analysis cleanly, firing `final_step` at the requesting point), `$stop` (pauses resumably), and aborts with a clear error on `$fatal`'s `E_PANIC` instead of retrying it as nonconvergence ([E-55](#)).

dcop.c (9) and **acan.c** (10)

- Fire `@(final_step)` at their successful end (an operating point fires both step events — a single point is first *and* last) ([E-53](#)).
- An AC job's operating-point phase carries the analysis name bit (only the name — adding the reactive `CALC_*` bits would wrongly enable integration at an op), so `analysis("ac")` holds through the whole AC run per LRM 4.6.1 and `@(initial_step("ac"))` fires in AC ([E-53](#)).

dctrcurv.c (150) + **include/ngspice/trcvdefs.h** (13)

- Generic `.dc @inst[param]` sweeps (a new sweep code): the DC sweep previously hardcoded `Vsource/Isource/Resistor/temperature`, so sweeping any other device parameter was a fatal error. The generic sweep resolves `@inst[param]` through the device's own parameter tables, refreshes per point via `DEVtemperature` (for OSDI exactly the `alter` path), nests with other sweep variables, and restores the original value afterwards ([E-62](#)).
- Fires `final_step` at the last sweep point; honors a `$finish` raised mid-sweep ([E-53](#), [E-55](#)).

noisean.c (37)

- A singular AC matrix during noise analysis crashed ngspice with a SIGABRT: the code ignored `NIacIter`'s return and the adjoint solve asserted on the unfactored matrix. It now aborts cleanly ("AC solution failed at ... Hz") ([E-56](#)).
- Honors deferred `$finish/$stop` raised at the noise operating point and fires `final_step` on success ([E-55](#), [E-53](#)).

span.c (31) + **cktspdum.c** (14)

- 1-port S-parameter analyses enabled: the "we need at least two ports" error was over-strict (a 1-port is a plain reflection measurement; the matrix machinery is N-general) — and it was hiding the `cadjoint` crash fixed in `dense.c` ([E-64](#)).
- `Rbase` published into the `sp` plot from port 1's `z0`, making the Touchstone writers work with no manual `let Rbase = 50` incantation ([E-64](#)).
- Stock NaN fixed in `donoise` noise-parameter extraction: the uncorrelated noise conductance `Gu` is analytically zero for fully-correlated single-source topologies, and floating-point rounding could land `sqrt(Ycor2 + Gu/Rn)`'s argument at $-10^{-18} \rightarrow \text{NaN}$ for the noise figure. Clamped to the physical range ≥ 0 — found by parity testing against an OSDI resistor that produced the exact textbook NF where the built-in returned NaN ([E-63](#)).

cktdisto.c (16)

- `.disto` silently reported zero distortion for OSDI devices — the distortion kernel needs higher-order Taylor coefficients the OSDI ABI cannot provide (first derivatives only), and such devices were skipped without a word. A prominent warning now names each affected OSDI device type ([E-62](#)).

4. Frontend — `src/frontend/`

plotting/pyplot.c (new) + **com_pyplot.c** (new) + **plotit.c, commands.c** (E-94)

A new interactive command, **pyplot**, plots simulated vectors with matplotlib — a Python counterpart to gnuplot. `ft_pyplot()` (`plotting/pyplot.c`) mirrors `ft_gnuplot()`: it writes a `<file>.data` table and a `<file>.py` matplotlib script and `system()`s `python3` (synchronously for a PNG, backgrounded for an interactive window). A new "pyplot" device arm in `plotit()` routes to it, so it accepts the same plot expressions as `plot/gnuplot`; the `com_pyplot()` wrapper mirrors `com_gnuplot()`, and `pyplot` is registered in both command tables. Two set variables tune it: `pyplot_terminal=png` renders headless (Agg) to a PNG, and `pyplot_python` picks the interpreter (default `python3`). Purely additive (E-94). The output file base name is optional (E-95): `com_pyplot()` treats the first word as a file name only when it is not a plot expression (no `(`, and not a vector by `vec_get()`), otherwise it defaults to `pyplot` — so `pyplot v(out)` plots directly (the command's minimum argument count was lowered from 2 to 1). `ft_pyplot()` also grew stacked subplots (set `pyplot_subplots=N` → `plt.subplots(nrows, 1, sharex=True)`, `N` traces per panel; `vs` stays the x-axis, so it is not a panel separator) and matplotlib style sheets (set `pyplot_style=<name>`, `dark` → `dark_background`, guarded) (E-98). The headless `pyplot_terminal` was extended from PNG-only to the vector export formats `svg` and `pdf` (set `pyplot_terminal=svg/pdf` → `fig.savefig(<file>.<fmt>)`, matplotlib picking the writer from the extension), and a figure size control was added (set `pyplot_figsize="W,H"` → `plt.subplots(..., figsize=(W, H))`; quote the value so ngspice keeps the comma) (E-99).

postcoms.c (414 diff lines) + **postcoms.h** + **commands.c** (16)

The Touchstone I/O subsystem (E-64, E-72):

- **wrsnp** (with `wrs2p` dispatching to the same handler): Touchstone v1 for any port count — the classic 2-port `S11 S21 S12 S22` column order preserved byte-identically, $N \geq 3$ in the spec's row-major layout with at most four complex pairs per line;
- writer options `wrsnp <file> [ri|ma|db] [s|y|z] [hz|khz|mhz|ghz]`, any combination in any order: MA/DB formats, Y/Z parameters (normalized to `Rbase`, $Y \cdot R / Z / R$, per the v1 spec), frequency units — the option line reflects every choice;
- **rdsnp**: reads any Touchstone v1 file into a new plot with a Hz frequency scale and complex vectors matching the `.sp` plot's conventions (MA/DB converted back, Y/Z de-normalized, the 2-port column order handled, `Rbase` published so imports round-trip) — measured data diffs against simulation in one `let` expression.

Why: E-63 showed the S-parameter *analysis* was exact but its results could not leave ngspice in the industry-standard format (`wrs2p` demanded a never-created `Rbase` vector), and nothing could bring VNA data in.

outitf.c (16)

- Integer operating-point variables recorded per point: `getSpecial` masked with `IF_REAL` only, so an integer opvar saved with `.save @n1[flag]` produced garbage in transient vectors (E-32).
- The plot-memory guard's message printed `%Id` — a Windows-only `printf` length modifier that is undefined behavior elsewhere and rendered the message as the numberless "memory required (Id Bytes)". Now `%zu`: real byte counts on every platform (E-77).

resource.c (27)

`ft_ckptspace()` — the "approaching max data size" warning (`RSS > 95%` of `RSS` + free RAM) — now honors `set no_mem_check` (the same opt-out the fatal output-size estimate in `outitf.c` respects; one variable governs both memory guards) and warns once per excursion above the threshold, re-arming below 90%, instead of repeating on every check through a long analysis (E-81).

display.c (1)

#undef NODEV before the local macro definition — the macOS SDK's <sys/param.h> defines an unrelated NODEV ([E-77](#)).

5. Parser and device registry — **src/spicelib/**

parser/ingtok.c (10)

A **.model** card override silently dropped its first parameter when the type name and the opening parenthesis had no space between them (`modname devtype(p1=v1 ...)`): INPgetNetTok's scan did not treat (as a token boundary, so the type token swallowed the paren and the first parameter name. Found while verifying event-function counters whose model-card overrides mysteriously kept defaults ([E-8](#)).

parser/ingmod.c (5)

`find_model_parameter()` dereferenced `*(device->numModelParms)`, which is NULL for built-ins that take no model cards (VCVS, CCCS, ...) — any *referenced* `.model m vcvs()` card segfaulted, with no OSDI involved at all (an ordinary MOS instance sufficed). This was the true root of the long-documented "module named like a built-in crashes" gotcha. One NULL guard; both shapes now produce clean, located errors ([E-76](#)).

devices/dev.c (51)

- Duplicate device-type registration warned and skipped: `osdi_add_device()` appended every descriptor unconditionally, and the model-card lookup scans front-to-back — so a module name duplicated across two loaded libraries silently resolved to the *first* registration (loading an updated model library gave the stale device with no hint), and a module named like a built-in corrupted model creation. Registration now checks the table case-insensitively and skips duplicates loudly ([E-76](#)).
- **pre_osdi** path dedup: loading an already-loaded file notes it and skips — with the remedy stated: *"restart ngspice to load a recompiled file"* ([E-76](#), [E-81](#)).

osdi/osdiparam.c (E-93)

Setting a fixed (**localparam**) OSDI parameter from the netlist used to be swallowed silently: `openvaf` exports every `localparam` as a parameter, its "given" flag is forced false, and the written value is overwritten by the parameter-init default — so `.model ... N=8` on a frozen structural width parameter changed nothing, with no feedback. `OSDIparam/OSDIiParam` now test the new `PARAM_FLAG_FIXED` descriptor flag (set by `openvaf`, a free bit of the parameter `flags` field) and emit *"parameter 'N' is a fixed (localparam) value and cannot be set from the netlist; ignored"* instead of silently dropping it ([E-93](#)).

6. Maths — **src/maths/**

dense/dense.c (11 substantive lines; the file also carries a whitespace/line-ending normalization from editing)

cadjoint() had no 1×1 base case: its cofactor loop allocated 0×0 and then negative-sized minors, killing ngspice with `malloc: can't allocate -8 bytes` — the crash that had been hiding behind the "at least two ports" guard in `span.c`. The adjugate of `[a]` is `[1]` (the empty minor's determinant is 1 by convention), making `cinverse` of a 1×1 equal $1/a$; a $100\ \Omega$ load on a $50\ \Omega$ port now writes a proper `.s1p` with $S_{11} = 1/3$ exactly ([E-64](#)).

sparse/spfactor.c (6)

The Markowitz-product overflow guards compare a double against `LARGEST_LONG_INTEGER` (= `LONG_MAX`), which is not exactly representable as a double; the conversion is now explicit — identical semantics, intentional and warning-free ([E-77](#)).

KLU/klu_multiply.c (16)

`SMPmultiply`'s KLU path passed `NULL` internal↔external ordering maps to `KLU_matrix_vector_multiply`, which dereferenced them unconditionally (`&NULL[n] = 0x14` for `n = 5`) and segfaulted — so `.option linesearch` crashed under `.option klu`. The line search is the only caller of `SMPmultiply` under KLU, so this path had never been exercised. A `NULL` map now means the identity ordering (`ext = i + 1`, since the KLU CSR is 0-based and the RHS/Solution vectors are 1-based), making the KLU matrix-vector product — and hence the [E-111](#) line-search residual merit — correct under KLU, with a merit sequence numerically identical to Sparse 1.3 ([E-112](#)).

KLU/klusmp.c (SMPcaSolve)

`SMPcaSolve` is the complex adjoint (transposed) solve used by noise and S-parameters. Its Sparse branch calls `spSolveTransposed`, but the KLU branch called the *non-transposed* `klu_z_solve` — silently wrong for any asymmetric matrix (every circuit with a transistor or controlled source), which is why `.noise` was disabled under KLU. It now calls `klu_z_tsolve` (transposed), so KLU noise matches Sparse exactly (including OSDI device models). This is the core of [E-113](#), which also removes the KLU refusal guards in `noisean.c` / `pzan.c` (single-ended pole-zero runs under KLU; balanced-output pole-zero keeps a targeted guard).

spicelib/analysis/cktsens.c

Sensitivity builds an auxiliary perturbation matrix `delta_Y` (holding $\partial Y / \partial p$) via `SMPnewMatrix`, which allocates a plain Sparse 1.3 matrix (`delta_Y->CKTkluMODE = 0`, no `SMPkluMatrix`). It is only *multiplied* against the solution (`SMPmultiply` → `spMultiply`), never factored, and `DEVbindCSC` always binds into `ckt->CKTmatrix`, not `delta_Y` — so it is correctly Sparse in every case. But two KLU-only setup blocks were gated on the main matrix's `CKTkluMODE`, so under `.option klu` they ran and dereferenced the `NULL delta_Y->SMPkluMatrix` — a segfault on every DC/AC `.sens` deck. The blocks now gate on `delta_Y->CKTkluMODE` (its own flag, `0`), keeping `delta_Y` Sparse under KLU exactly as it already is under the default solver, while the main `Y` matrix stays KLU. DC and AC sensitivity now match Sparse exactly ([E-114](#)).

spicelib/analysis/distoan.c

Distortion is a complex analysis (Volterra series solved at the harmonic / intermodulation frequencies via `NIIdIter` → `SMPcSolve`), but `distoan.c` had no KLU code at all: under `.option klu` the KLU matrix stayed in *real* mode, the complex solves ran against an unconverted matrix, and every harmonic came back zero — a silent wrong answer. The fix mirrors `acan.c`: before the solve loop it converts the matrix to complex (`DEVbindCSCComplex` per device + `KLUmatrixIsComplex`), and on exit converts back to real (`DEVbindCSCComplexToReal`) so a later real analysis in the same session is unaffected. KLU distortion now matches Sparse bit-for-bit (single-tone, two-tone intermodulation, and OSDI models) ([E-115](#)).

7. Build system

xspice/icm/makedefs.in (4)

The XSPICE codemodel (.cm) link rule hardcoded the pre-macOS-10.3 idiom `-bundle -flat_namespace -undefined suppress`, deprecated loudly by the modern linker on all 14 codemodel links (CI macOS builds included). Now `-bundle -undefined dynamic_lookup` — the modern spelling for plugins whose host symbols resolve at load time (E-77).

Autotools regeneration (the ~100 **Makefile.in** files + **config.h.in**)

Regenerated by `./autogen.sh` with libtool 2.5.4 during the zero-warning work — a build tree whose configure predates libtool 2.4.7 selects the deprecated darwin `-undefined suppress` flag whenever `MACOSX_DEPLOYMENT_TARGET` is unset. No hand-written content (E-77).

8. Per-enhancement index

Every enhancement that touched ngspice, oldest first:

Enhancement	ngspice files	One line
E-1	osdi.h, osdidefs.h, osdiitf.h, osdiregistry.c, osdiload.c, osdisetup.c, dctran.c (+accept path)	absdelay(): synthetic delay rows + waveform history
E-6	osdi.h, osdidefs.h, osdiitf.h, osdiregistry.c, osdiload.c	last_crossing(): history + interpolated crossings
E-8	parser/ingptok.c	.model card first-parameter drop fix
E-24	osditrunc.c	\$discontinuity next-step clamp
E-25	osdi callbacks (get_simparams)	expose analysis_name + simulator simparams
E-32	outitf.c	integer opvars recorded per point
E-42	osdinoise.c	coherent same-named noise grouping (LRM 4.6.4)
E-45	osdi.h (ABI 0.5), osdisetup.c	Verilog-A nodesets applied to the solver
E-51	osdi.h (ABI 0.6), osdiacld.c	full ac_stim AC-RHS injection
E-53	dctran.c, dcop.c, dctrcurv.c, acan.c, noisean.c, osdiload.c	@(final_step) + analysis-name phase bits
E-54	osdi.h (ABI 0.7), osdinoise.c, osdiload.c, osdiacld.c, osdiregistry.c	node-free complex noise factors, stride-2 pairs
E-55	osdiaccept.c (new), osdiload.c, osditrunc.c, osdiinit.c, dctran.c, dctrcurv.c, osdiitf.h, Makefile.am	\$finish/\$stop/\$fatal honored; \$discontinuity step rejection
E-56	noisean.c, osdisetup.c	noise SIGABRT fix; setup-rejection diagnostics
E-62	dctrcurv.c, trcvdefs.h, cktdisto.c	generic .dc @inst[param] sweeps; .disto warning
E-63	span.c	donoise NaN clamp (stock defect, found by parity)
E-64	span.c, cktspdum.c, dense.c, postcoms.c/.h, commands.c	Touchstone export, auto-Rbase, 1-port .sp, cadjoint 1x1
E-72	postcoms.c/.h, commands.c	Touchstone round 2: MA/DB/Y/Z/units + rdsnp reader

Enhancement	messspice files	One line
E-76	dev.c, inpgmod.c	duplicate-registration warning, load dedup, stock .model segfault fix
E-77	osdidefs.h, display.c, outitf.c, spfactor.c, makedefs.in (+autotools regen)	zero-warning build, 33 → 0
E-80	osdiinit.c	dtemp instance-parameter alias
E-81	resource.c, dev.c	memory-warning opt-out + once-per-excursion; reload-note hint
E-93	osdi.h, osdiparam.c	warn (not silently ignore) when a netlist sets a PARA_FLAG_FIXED (localparam) OSDI parameter
E-94	plotting/pyplot.c (new), com_pyplot.c (new), plotit.c, commands.c	new pyplot command — plot vectors with matplotlib (a Python gnuplot)
E-95	com_pyplot.c, commands.c	make the pyplot output file name optional (defaults to pyplot)
E-98	plotting/pyplot.c	pyplot stacked subplots (pyplot_subplots=N) + matplotlib style sheets (pyplot_style)
E-99	plotting/pyplot.c	pyplot vector export formats (pyplot_terminal=svg/pdf) + figure size (pyplot_figsize)
E-110	optdefs.h, tsdefs.h, cktsopt.c, cktntask.c	.option errpre-set=conservative moderate liberal — one knob for a coordinated tolerance/robustness set; explicit options override regardless of order (moderate = historical defaults)
E-111	optdefs.h, tsdefs.h, cktdefs.h, cktsopt.c, cktdjob.c, cktntask.c, cktdtest.c, niiter.c	.option linesearch — globalized (damped) Newton via Armijo backtracking on a new KCL-residual merit $F = G \cdot x - b$; result-neutral, off by default
E-112	maths/KLU/klu_multiply.c	KLU support for .option linesearch — SMPmultiply's KLU path passed NULL ordering maps that klu_matrix_vector_multiply dereferenced (SIGSEGV); NULL now means identity ordering. Line search verified merit-identical KLU vs Sparse
E-113	maths/KLU/klusmp.c, spicelib/analysis/noisean.c, pzan.c	KLU support for noise + single-ended pole-zero — SMPcaSolve's adjoint KLU branch used the non-transposed solve (silently wrong noise on asymmetric matrices); now klu_z_tsolve. Guards removed; balanced-output pz keeps a targeted guard

Enhancement	spice files	One line
E-114	spicelib/analysis/cktsens.c	KLU support for sensitivity — the auxiliary perturbation matrix <code>delta_Y</code> is Sparse, but two KLU setup blocks gated on the <i>main</i> matrix's flag dereferenced its NULL <code>SMPkluMatrix</code> (segfault on every DC/AC <code>.sens</code>); now gated on <code>delta_Y</code> 's own flag. DC/AC <code>.sens</code> match Sparse exactly
E-115	spicelib/analysis/distoan.c	KLU support for distortion — <code>.disto</code> is a complex analysis but <code>distoan.c</code> had no KLU code, so under KLU the matrix stayed real and every harmonic came back zero (silent wrong answer); now converts real↔complex around the solve loop like <code>acan.c</code> . Matches Sparse bit-for-bit
E-116	osdi/osdisetup.c	KLU wrong-DC fix — an OSDI internal node with no Jacobian entry (a decoupled ground reference) was given an all-zero solver row that made the KLU matrix structurally singular; now tied to ground. Fixes the <code>groundcontrib</code> and <code>hierbranch</code> KLU_XFAILs
E-117	configure.ac (+configure), spicelib/analysis/dcpss.c	Periodic steady state (PSS) productionized — built by default (was experimental <code>--enable-pss</code> , so <code>.pss</code> was unimplemented in shipped builds); ~230 lines of shooting-loop trace routed through <code>setngdebug</code> (232 → 31 lines by default); fail-fast KLU guard (PSS hangs under KLU) directing <code>.pss</code> to <code>.option sparse</code>
E-118	spicelib/analysis/dcpss.c	PSS runs under KLU — the shooting loop hung under KLU (<code>klu_refactor</code> 's reused pivots inflated the truncation error into a ~20M-step timestep explosion); forcing a full re-factor (NISHOULDREORDER) each PSS step makes KLU converge to the same result as Sparse. E-117's Sparse-only guard removed
E-119	pssdefs.h, spicelib/analysis/dcpss.c	Retain the PSS periodic operating point — PSS sampled the node voltages per period for its DFT then freed them, and never captured device states; now the voltages and <code>CKTstate0</code> states are captured per sample and retained on the PSSan job (with frequency + dims) as the substrate for the periodic small-signal suite (PAC/pnoise/PXF). Self-checks that the retained samples reproduce the fundamental

Enhancement	ngspice files	One line
E-120	spicelib/analysis/dcpss.c	Periodic small-signal Jacobian harmonics — walk the retained operating point, re-linearize each sample (CKTload MODEINITSMSIG) and stamp $G+jC$ (CKTload $\omega=1$), read the osc-node diagonal $\rightarrow g(t), c(t)$, DFT to harmonics. The blocks the PAC conversion matrix is built from. (Fixed a complex-mode-not-set bug where $C(t)$ accumulated across samples.) Verified: RC gives $G=1/R1, C=C1$, no harmonics
E-121	spicelib/analysis/dcpss.c	PAC conversion-matrix engine — extend E-120 from the osc diagonal to every matrix nonzero, complex-DFT to harmonics G_k, C_k , assemble the $(2M+1)N$ harmonic conversion matrix $H_{\{nm\}}=G_{\{n-m\}}+j\omega_m \cdot C_{\{n-m\}}$, inject a unit current at the osc node in sideband 0 and solve by dense complex LU (pss_solve), report the sideband conversion gains. The numerical heart of PAC/pnoise/PXF. Verified: linear RC gives sideband-0 = AC driving-point ‘
E-122	include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c	.pac command (periodic AC) — the user-facing analysis on the E-121 engine. .pac Fguess StabTime OscNode Points Harmonics SC_iter Steady_coeff <dec oct lin> Npts Fstart Fstop runs PSS then sweeps the input frequency, solving the conversion matrix at each point and emitting the 0-th-sideband node responses as a complex PAC Analysis plot (print/plot/wrdata). Reuses the PSS analysis via a PSSdoPAC flag; engine factored into pac_extract_harmonics (once) + pac_solve_at (per freq). Verified: swept sideband-0 $ b(f) ==$ analytic AC driving-point $ Z(f) $ to $1.6e-7$ across 10 kHz–1 MHz

Enhancement	ngspice files	One line
E-123	include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c	finish .pac — (1) source-referenced stimulus: capture the netlist AC-source RHS from CKTAcLoad as the sideband-0 stimulus B_0 (a periodic-AC transfer/conversion gain), unit-current-at-osc-node fallback; (2) multi-sideband output: optional trailing maxsideband Ksb emits every conversion sideband $f_{in+k} \cdot f_0$ ($k=-Ksb..Ksb$) as its own vector — sideband 0 keeps the node name, others <code><node>_usb<k>/<node>_lsb<k></code> via <code>IFnewUid</code> . Verified: RC with V1 AC 1 gives sideband-0 = low-pass transfer $1/\sqrt{1+(2\pi fRC)^2}$ (0.998→0.157) with <code>b_usb1/b_lsb1 ~0</code> (no conversion)
E-124	include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c	.pnoise command (periodic noise) — fold each device's noise through the conversion matrix. <code>pac_solve_adjoint</code> solves $H^T \Psi = e_{\{out,0\}}$; <code>pnoise_sweep</code> loads the sideband-k adjoint into <code>CKTrhs/CKTirhs</code> and calls the existing device noise routines (<code>NevalSrc</code> , <code>OSDI load_noise</code>) once per sideband, accumulating $\Sigma_k S \cdot \Delta\Psi_k ^2$ — reuses every device noise model via a minimal local <code>NOISEAN</code> context. .pnoise <code><pss> OutNode InSrc <dec oct lin> Np Fstart Fstop</code> ; emits <code>onoise_spectrum/inoise_spectrum</code> . Verified: linear RC reduces exactly to .noise ($4kTR/(1+(2\pi fRC)^2)$), matches the .noise reference to every digit)
E-125	include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c	.pxf command (periodic transfer function) — the adjoint of PAC, completing the <code>PSS→PAC→Pnoise→PXF</code> suite. <code>pxf_sweep</code> solves $H^T \Psi = e_{\{out,0\}}$ per frequency and dots each sideband block with the AC-source pattern $B_0 \rightarrow xf_k = \Sigma_j \Psi_k(j) \cdot B_0(j)$, the input→output transfer at each sideband. .pxf <code><pss> OutNode <dec oct lin> Np Fstart Fstop [maxsb]</code> ; emits <code>xf + xf_usb<k>/xf_lsb<k></code> . Verified: by the identity $(H^{-1}B)_{out} = (H^{-T}e_{out})^T B$ the sideband-0 transfer is bit-identical to the PAC response (0.998→0.157 low-pass), conversion sidebands $\sim 2e-16$

Enhancement	ngspice files	One line
E-126	include/ngspice/pssdefs.h, spicelib/analysis/psssetp.c, spicelib/parser/inp2dot.c, spicelib/analysis/dcpss.c	cyclostationary noise — <code>.pnoise ... cyclo</code> evaluates each device's noise PSD $S(t)$ at every PSS sample's bias (CKTload per sample) and folds it through the time-domain adjoint transfer $A_s(j) = \sum_k \Psi_k(j) e^{j2\pi k s/P}$, averaging: $\text{onoise}(f) = (1/P) \sum_s S(t_s) \cdot \Delta A_s ^2$. Reuses the device noise routines. Verified: (1) reduces exactly to <code>.noise</code> on the linear RC (bias-independent thermal $\rightarrow$ Parseval); (2) a flicker resistor carrying the RC current gives $\text{onoise} \cdot f = R^2 \cdot KF \cdot \langle I^2 \rangle = 4.88e-10$ (uses the period-average $\langle I^2 \rangle$, matched to 5 digits)
E-127	include/ngspice/{optdefs,tskdefs,cktdefs}.h, spicelib/analysis/{cktsopt,cktnntask,cktdojob,cktdontest}.c, maths/ni/niiter.c	pseudo-transient continuation — <code>.option pntc</code> continues $f(x)=0$ in a backward-Euler pseudo-transient $f(x) + G_{ps} \cdot (x - x_{\text{prev}}) = 0$ and marches $d\tau$ small $\rightarrow$ large ($G_{ps} \rightarrow 0$). G_{ps} diagonal via the <code>gmin</code> path; the $G_{ps} \cdot x_{\text{prev}}$ RHS coupling (added in <code>NIiter</code>) makes each step a stable-trajectory move, not a static <code>gmin</code> step. New <code>pseudo_transient</code> in the CKTop cascade, off by default. Verified under KLU+Sparse: result-neutral on normal circuits; on a stiff no-limiting exp, reaches the correct DC 0.837922 V where plain Newton lands on a spurious 70.5 V (21/21)
E-128	include/ngspice/{optdefs,tskdefs,cktdefs}.h, spicelib/analysis/{cktsopt,cktnntask,cktdojob}.c, spicelib/analysis/dctran.c	LTE-based dynamic integration-order control — <code>.option dynorder</code> selects the Gear order per step from the local-truncation-error limit instead of the stock $1 \leftrightarrow 2$ toggle, so orders 3–6 (already coded in <code>NIcomCof</code> but never used) are actually exercised. The selector compares the raw LTE-limited step (uncapped CKTrunc) at the current order and its ± 1 neighbours and moves with hysteresis ($1.2\times$), a settling hold after each change (let the BDF divided differences rebuild), and an order-dependent step-growth cap ($2\times$ at order $\leq 3 \rightarrow 1.3\times$ at 6). Off by default; bounded by <code>maxord</code> ; inert at the default <code>maxord=2</code> . Stiff-transient guards (post-breakpoint order hold + leaky-bucket rejection-rate order drop) keep it from collapsing the step on a violent slew. Verified under KLU+Sparse: RC decay $3\text{--}5\times$ fewer steps at matched accuracy with monotone error; smooth RLC ringdown $8.9\times$ fewer steps AND more accurate (0.13 % vs 0.34 %); nonlinear rectifier matches stock to 5 sig figs; the transistor-level $\mu A741 \pm 5$ V slew completes under <code>dynorder</code> and matches fixed Gear-2

Enhancements	ngspice files	One line
E-129	frontend/outitf.c	sweep progress bar — the throttled Reference value status line (redrawn in place every 0.25 s during a sweep) now carries a live progress bar + percentage, e.g. Reference value : 5.91926e-04 [=====] 74%. outp_progress_frac() computes the 0-1 fraction per analysis: transient from (CKTtime-TSTART)/(TSTOP-TSTART), AC/noise from the frequency's linear/log position in the band, DC from the accepted-point count over the nested step-count product; analyses with no span (op, ...) keep the plain line. Fixed-width (no stale chars), stdout status-line only (never in the rawfile/wrdata), same !ft_norefprint && !cp_background gating. Verified 22/22: printed % matches the analytic sweep fraction to 0.5 % across tran/AC/DC/noise; op shows no bar
E-130	frontend/com_optimize.{c,h}, commands.c, options.c, include/ngspice/ftext.h, spicelib/analysis/cktdojob.c, frontend/outitf.c	built-in Nelder-Mead optimizer — optimize -param <name> <init> <lo> <hi> [-param ...] -analysis <cmd> -minimize <expr> [-maxiter N] [-tol T] [-verbose] varies device/alter parameters, re-runs an analysis, and minimizes a scalar objective expression via a derivative-free downhill simplex in normalized [0,1] space (scale-invariant across orders-of-magnitude params). Sub-commands dispatched synchronously through cp_coms (not the deferring re-entrant cp_ev loop); the hundreds of inner analyses are silenced via a new ft_optimizing flag gating the banner/row-count/E-129 progress bar at source (-verbose to show). Verified 9/9 against analytic optima: DC divider R1→2333.3 Ω, AC low-pass R1→2756.6 Ω, 2-D compound objective R1=3k/R2=2k exactly

Enhancement	ngspice files	One line
E-131	frontend/com_checkpoint.{c,h}, commands.c, com_commands.h, Makefile.am, spicelib/analysis/dctran.c, include/ngspice/cktdefs.h	transient checkpoint / restart — new <code>savestate <file> / loadstate <file></code> commands serialize the full transient integration state (solution vector <code>CKTrhsOld</code> , device state history <code>CKTstates[]</code> , time/step/order/mode, pending breakpoints) to a binary file and resume it, including in a fresh process — so a long run survives a crash, splits across sessions, or moves between machines (stock ngspice could only resume in memory). <code>DCTran()</code> gains a <code>CKTcheckpoint</code> -gated branch that opens a fresh output plot (no live plot to 666-relink across a reload), initializes the XSPICE breakpoint markers, and fixes up the breakpoint list before jumping into the stepping loop; the rhs length is keyed off <code>SMPmatSize+1</code> (not <code>CKTmaxEqNum</code>). A stored signature rejects a mismatched circuit; Sparse-solver only (KLU rejected with a clear message). Verified 19/19: resumed waveform bit-identical to an uninterrupted run for RC-step/pulse/diode built-ins and $\sim 2e-7$ for a compiled OSDI diode, across a separate process

Enhancement	ngspice files	One line
E-132	spicelib/analysis/dcpss.c, spicelib/parser/inp2dot.c, spicelib/analysis/psssetp.c, include/ngspice/pssdefs.h	periodic S-parameters (.psp) — small-signal scattering parameters around a PSS operating point, including conversion between the input frequency and its sidebands $f_{in+k} \cdot f_0$ (mixers / switched circuits, where a static-DC .sp cannot see the conversion). Sits on the PSS→conversion-matrix suite (E-117–126): after PSS, psp_sweep excites each RF port (portnum/z0, the .sp framework) by driving its branch source (V=1, like .sp's VSRCspupdate) through the shared $(2M+1)N$ conversion matrix, reads per-sideband port waves in the same Kurosawa power-wave convention, and forms $S^{(k)} = B^{(k)} \cdot A^{-1}$ (dense-complex cinverse/cmultiply). pac_solve_at's matrix assembly factored into a reusable pac_build_matrix; PSSdoPSP flag + psp PSS param + dot_psp card. Because $S = B \cdot A^{-1}$ is excitation-basis-invariant, sideband 0 reduces exactly to .sp for a time-invariant network. Runs under both linear solvers (the conversion matrix is a standalone dense LU; PSS runs under both since E-118). Verified 8/8: sideband-0 matches .sp to $\sim 1e-16$ for 1/2/3-port resistive + reactive networks (magnitude and phase) incl. OSDI Verilog-A devices (G + reactive ddt stamps), conversion sidebands correctly ~ 0

Enhancement	ngspice files	One line
E-133	frontend/com_qpss.{c,h}, commands.c, com_commands.h, Makefile.am	quasi-periodic (two-tone) steady state (qpss) — qpss <expr> <f1> <f2> [periods] [maxorder] computes the two-tone steady-state spectrum: every mixing product $k_1 \cdot f_1 + k_2 \cdot f_2$ including third-order intermodulation (IM3 at $2f_1 - f_2$ / $2f_2 - f_1$) that single-tone AC/PSS cannot show. For commensurate tones (common beat $fb = \gcd(f_1, f_2)$) it runs an ordinary transient over a few beat periods to reach steady state, then evaluates the Fourier coefficient directly at each exact intermod frequency (a direct DFT, exact — no FFT-bin rounding) and labels it by the 2-D index (k_1, k_2). Deliberately not a slow beat-frequency shooting PSS; a front-end command driving a transient, so solver-independent and works with built-in and OSDI devices. Verified 11/11: analytic fundamentals/IM3/3f of a cubic, no even-order products, the 3:1 IP3 slope law (fund $\times 2$, IM3 $\times 8$ per $2\times$ drive), beat-frequency derivation, and an OSDI cubic matching the built-in

Enhancement	ngspice files	One line
E-134	spicelib/analysis/dcpss.c, frontend/com_hb.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h	Harmonic Balance (hb <f0> <K>) — solves the periodic steady state in the frequency domain by Newton (each node a truncated Fourier series), instead of integrating in time; the real analysis behind ngspice's unimplemented WITH_HB stub. Residual $F_k = I_{R,k}(V) + [dq/dt]_k - Is_k = 0$ with the E-121 $(2K+1)N$ conversion matrix as the exact Jacobian. Per iteration: inverse-DFT $V \rightarrow v(t_s)$; drive DC+AC device loads at those voltages for the resistive current $I_R = G*v - b$ (companion b saved before acLoad, so it's the ACTUAL current not the tangent $G*v$), and $G(t)/C(t)$; DFT; dense complex Newton (pss_csolve). Nonlinear reactive with NO charge extraction: $dq/dt = C(v)*v'$, so the reactive current is the conversion matrix's jwC term applied to V. Solver-independent (KLU + Sparse): the dense complex Newton is HB's own; the sparse solver only <i>reads</i> $G(t)/C(t)$ off the device matrix, so hb_extract carries the same <code>#ifdef KLU</code> complex-CSC binding (<code>DEVbindCSCComplex</code>) as the PAC extraction, and com_hb copies the task's KLU mode before building so a bare hb honours <code>.option klu</code> — verified bit-identical under both. Built-in + OSDI. Verified 8/8 vs transient/fourier, quadratic convergence: nonlinear R (analytic 3rd harmonic), R+C, a real diode rectifier (junction limiting), OSDI varactor $Q(v)$ 2nd harmonic, KLU==Sparse parity

Enhancement	ngspice files	One line
E-135	spicelib/analysis/dcpss.c, examples/hb_examples/verify_hb.py	HB source-stepping continuation — makes E-134 Harmonic Balance robust on strongly-driven circuits where a cold full-strength Newton diverges ($ F \rightarrow 1e69$). HBanalyze's Newton loop is wrapped in an adaptive homotopy: every independent source scaled by $\lambda: 0 \rightarrow 1$, each level warm-started from the last converged V. The residual becomes $F = I_R + I_C - \lambda \cdot I_s$; on level success V is checkpointed and $d\lambda$ grows ($\times 1.7$), on failure (Newton exhausted, residual non-finite, or singular Jacobian) V is restored and $d\lambda$ halves; $d\lambda < 1e-5 \rightarrow$ reported as no reachable steady state. First level is full strength ($d\lambda=1$) so easy circuits converge at $\lambda=1$ immediately, bit-identical to the plain solve. Automatic (no new syntax); set <code>hb_verbose</code> shows the λ ramp, summary reports iterations + continuation steps. Verified 9/9: a strongly-driven diode rectifier (5 V into 20 Ω , sharp $I_S=1e-14$) diverges cold but converges in 3 continuation steps, matching transient fourier to $<0.1\%$ for DC/ $f_0/2f_0$; the 8 existing HB checks stay bit-identical

Enhancement	ngspice files	One line
E-136	spicelib/analysis/dcpss.c, frontend/com_qpss.c, frontend/commands.c, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpss_hb.py	two-tone Harmonic Balance (qpss <expr> <f1> <f2> hb [K1] [K2]) — the TRUE, incommensurate-capable quasi-periodic steady state, a frequency-domain HB engine alongside the E-133 transient qpss (which is unchanged, commensurate-only). Each node is a 2-D Fourier series $v(t) = \sum \sum V_{k1,k2} e^{j(k1\omega1+k2\omega2)t}$; QPSShb samples devices on a 2-D phase grid ($\theta1, \theta2$) — time never appears, so incommensurate tones (no beat period) just work — and 2-D DFTs to the conversion matrix $H_{(n),(m)} = G_{n-m} + j\omega_m \cdot C_{n-m}$ (qp_build_matrix), Newton-solved by pss_csolve with the E-135 source stepping. Sources captured by an oversampled least-squares APFT ($\Gamma^H \Gamma$) $I_s = \Gamma^H b$ (a square Vandermonde is catastrophically ill-conditioned past a few harmonics). Reactive needs NO charge extraction ($dq/dt = C(v)v'$); the converged operating point is retained (qpss_hb_saved) for qpac (E-137). Solver-independent (KLU + Sparse) — same <code>#ifdef KLU</code> complex-CSC binding as PAC/HB. Built-in + OSDI. Verified 7/7: analytic cubic $ IM3(2,-1) / 3rd(3,0) =3$, even products ~0, 3:1 IP3 slope, incommensurate 2 tones (E-133 cannot), HB==transient (commensurate), KLU==Sparse; E-133 verify_qpss.py stays 11/11

Enhancement	ngspice files	One line
E-137	spicelib/analysis/dcpss.c, frontend/com_qpac.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpac.py	two-tone small-signal QPAC (qpac <f_in>) — completes the quasi-periodic gap: the two-tone analogue of PAC. Run after qpss ... hb, QPACanalyze injects a small signal at f_in around the retained QPSS operating point (qpss_hb_saved) and reports the response at every sideband f_in+k1·f1+k2·f2, mixing it through the SAME 2-D conversion matrix $H_{\{(n),(m)\}} = G_{\{n-m\}} + j\omega_m \cdot C_{\{n-m\}}$ (qp_build_matrix at f_in) that the QPSS Newton used as its Jacobian. Stimulus placed in the (0,0) sideband — a netlist AC-source RHS B0 (captured bias-independent at the op-point) or a unit-current fallback — one dense pss_csolve. Adds no device evaluation → solver-independent (KLU + Sparse) by construction. Verified 7/7: reduce-to-AC (pump→0) direct (0,0) = plain .ac response = R, sidebands vanish), v ² -pump conversion ratio $ (1,1) / (2,0) =2$, equal-tone symmetry, clean no-op-point error, KLU==Sparse; E-136 (7/7) + E-133 (11/11) unaffected

Enhancement	ngspice files	One line
E-138	spicelib/analysis/dcpss.c, frontend/com_qpnoise.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpnoise.py	two-tone small-signal QPnoise (qpnoise <output_node> <f_in>) — quasi-periodic noise, the two-tone analogue of pnoise (E-124), on the retained qpss ... hb operating point. Reports output + input-referred noise density at f_in, folding every device's noise over all sidebands $f_{in} + k1 \cdot f1 + k2 \cdot f2$ (mixer/PA noise conversion invisible to a static .noise). QPnoiseAnalyze biases the devices at the QPSS operating point, then qp_solve_adjoint transposes the 2-D conversion matrix (qp_build_matrix) and solves $H^T \Psi = e_{\{out, (0,0)\}}$ — one adjoint gives the transimpedance from every (node, sideband) to the output; each device's DEVnoise computes $S \cdot \Psi ^2$ (reading the transfer from CKTrhs/CKTirhs) and the sum over all Nh harmonics is the folded onoise; input-referred via the QPAC gain. Reads only retained data → solver-independent (KLU + Sparse) by construction. With no pump the conversion matrix is block-diagonal so only (0,0) survives [?] reduces to .noise. Verified 6/6: reduce-to-noise (pump→0 [?] onoise == plain .noise == 4kTR exactly), conversion active under pump, inoise=onoise/gain ² , clean no-op-point error, KLU==Sparse; E-133 (11/11) + E-136 (7/7) + E-137 (7/7) unaffected

Enhancement	ngspice files	One line
E-139	spicelib/analysis/dcpss.c, frontend/com_qpnoise.c, frontend/commands.c, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpnoise.py	cyclostationary QPnoise (qpnoise <output_node> <f_in> cyclo) — upgrades E-138 to a time-varying device PSD. Under a two-tone pump a device's bias swings over the period (a diode's shot noise $2qI_D(t)$ spikes when it conducts), so QPnoiseAnalyze gains a cyclo branch using the identity $\text{noise} = (1/P) \cdot \sum_s$ $S(t_s) \cdot A_s ^2$, where $A_s(j) =$ $\sum_{\{(k1,k2)\}} \Psi_{\{(k1,k2)\}}(j) \cdot e^{j2\pi(k1$ $s1/P1+k2 s2/P2)}$ is the inverse 2-D DFT of the adjoint transfers (the time-domain transimpedance at phase sample s). Each sample re-biases the devices at the retained op-point (qp_synth, P1/P2 now stored in qp_harm) with per-sample junction settling (the E-134 MODEINITFLOAT walk — else a limited diode reports a stale bias and the "cyclostationary" result collapses onto the stationary one), evaluates $S(t_s)$, folds through A_s , and averages. By Parseval reduces to the stationary sum (and .noise) when S is constant. Verified 10/10 (6 E-138 + 4 cyclo): cyclo reduce-to-noise, Parseval (bias-independent thermal PSD [?] cyclo==stationary even under pump), a hard-pumped diode where cyclo differs from stationary by $\sim 8\times$ (switching-mixer cyclostationary enhancement, visible only with the settling), cyclo KLU==Sparse; E-133/136/137 unaffected

Enhancement	ngspice files	One line
E-140	spicelib/analysis/dcpss.c, frontend/com_hbosc.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, exam- ples/phasenoise_examples/verify_phasenoise.py	oscillator phase noise (hbosc <oscnod> <K> [fguess] [tstab] + phasenoise <fstart> <fstop> [pts]) — closes the periodic/phase-noise gap with the phase-noise piece. HBOSCanalyze is an autonomous harmonic balance: an oscillator has no source, so $F(V)=I_R+[dq/dt]=0$ is solved for the harmonics AND the unknown oscillation frequency ω_0 . The conversion matrix $dF/dV=H$ is singular (right null space = phase mode $u_k=j_k V_k$, since the oscillator's phase is free), so Newton runs on the bordered system $[H, \partial F/\partial \omega_0; u^*, 0]$ (nonsingular by bordering, $\partial F/\partial \omega_0=I_C/\omega_0$, gauge row u^*), transient-seeded (hbosc runs a short transient from the deck's .ic, reads amplitude + zero-crossing frequency), converging quadratically (LC osc: 4 iters to $ F =3e-12$; inductors fit the $G+j\omega C$ matrix via branch-current MNA). PhaseNoiseAnalyze builds the adjoint of H at OFFSET df with the unit at the carrier sideband ($m=1$) and folds device noise (DEVnoise); as $df \rightarrow 0$ the limit-cycle matrix goes singular through the phase mode so the transimpedance diverges as $1/df \rightarrow$ folded noise $1/df^2$, normalized to carrier power $2 V_1 ^2 \rightarrow L(df)$ in dBc/Hz. Verified 8/8 on an LC oscillator: autonomous HB converges to f_0 +describing-function amplitude, $L(df)$ has the -20 dB/dec skirt near carrier flattening into the noise floor at a physical level (≈ -147 dBc/Hz @1kHz), thermal scaling $L \propto T$ ($2 \times T \rightarrow +3$ dB exactly, pinning the absolute), clean no-op-point error, KLU==Sparse; pnoise (9/9) + qpnoise (10/10) unaffected

Enhancement	ngspice files	One line
E-141	spicelib/analysis/dcpss.c, frontend/com_qpxf.{c,h}, commands.c, com_commands.h, Makefile.am, include/ngspice/cktdefs.h, examples/qpss_examples/verify_qpxf.py	two-tone small-signal QPXF (qpxf <output_node> <f_in>) — the quasi-periodic transfer function, the ADJOINT of QPAC (E-137), completing the two-tone small-signal suite (QPSS→QPAC→QPnoise→QPXF ≡ PSS→PAC→pnoise→PXF). QPXFanalyze runs one adjoint solve $H^T \Psi =$ $e_{\text{out}, (0,0)}$ (qp_solve_adjoint, reused from QPnoise E-138) and dots each sideband block of Ψ with the netlist AC-source pattern B0 (the QPAC stimulus, captured at the op-point) → the transfer $H_{\{(k1,k2)\}} =$ $\sum_j \Psi_{\{(k1,k2), j\}} \cdot B0_j$ from an input at every sideband $f_{\text{in}} + k1 \cdot f1 + k2 \cdot f2$ to the output, all from ONE adjoint. By the reciprocity identity $(H^{-1}B)_{\text{out}} = (H^{-T}e_{\text{out}})^T B$ the sideband- $(0,0)$ transfer is bit-identical to the QPAC response (the PXF↔PAC cross-check, E-125). Reads only retained data → solver-independent (KLU + Sparse). Verified 6/6: reciprocity (QPXF(0,0)==QPAC(0,0) bit-identical), conversion-sideband reciprocity, reduce-to-XF (pump→0 ? $(0,0)$ =plain transfer=R, sidebands vanish), clean no-op-point error, KLU==Sparse; QPSS 11/11 + QPSS-HB 7/7 + QPAC 7/7 + QPnoise 10/10 unaffected

Enhancement	ngspice files	One line
E-142	spicelib/analysis/dcpss.c, frontend/com_qpac.{c,h}, com_qpnoise.c, com_qpxf.c, commands.c, include/ngspice/cktdefs.h, exam- ples/qpss_examples/verify_qpss_sweep.py	input-frequency sweep for the two-tone small-signal analyses. qpac/qpnoise/qpxf gain a <dec oct lin> N fstart fstop sweep of f_in that emits a plottable ngspice plot (conversion gain / noise figure / image-rejection curves vs frequency), matching how .ac/.pnoise/.pxf sweep. QPACsweep/QPnoiseSweep/QPXFstep f_in and reuse the exact single-frequency solve per point: qpac records per-node (0,0) response , qpnoise records onoise_spectrum/inoise_spectrum, qpxf records xf (in-band (0,0)) + xf_conv (total conversion $\sqrt{\sum H_{\{sb \neq 0\}} ^2}$). The analysis fills plain arrays; the front-end builds the plot with a frequency scale via the nutmeg vector API (plot_alloc/dvec_alloc/vec_new) — the correct layer for a front-end command (the analysis OUTp framework dereferences a live analysis job's JOBtype, which a front-end command lacks → was crashing). Shared helpers qp_steptype/qp_sweep_maxpts/qp_emit_plot. Verified 5/5: swept value at 0.3G == single-frequency qpac/qpxf/qpnoise (machine precision), reactive roll-off vs f_in, correct point count; single-frequency QPAC 7/7 + QPnoise 10/10 + QPXF 6/6 unaffected

Enhancement	ngspice files	One line
E-143	frontend/com_optimize.c, frontend/commands.c, exam- ples/optimize_examples/verify_optimize.py, exam- ples/optimize_examples/optimize_lsq_demo.c	gradient least-squares curve fitting for optimize . The built-in optimizer (E-130, derivative-free Nelder-Mead on a scalar <code>-minimize</code>) gains a least-squares mode: one or more weighted <code>-target <expr> <value> [<weight>]</code> measurements, optionally spread over several <code>-analysis</code> stages (each <code>-analysis</code> opens a stage; following <code>-targets</code> attach to it, so a single fit can combine e.g. a DC operating point AND an AC response), are fitted with Levenberg-Marquardt — a forward/backward finite-difference Jacobian J , normal equations $A=J^T J$, $g=J^T r$, damped solve $(A+\lambda \cdot \text{diag}(A)) \delta = -g$ via a small partial-pivot Gauss solver, standard λ up/down loop. Exploiting the sum-of-squares structure it converges in far fewer analysis runs than the simplex (27 vs 67 evals on a two-target RC fit). <code>-method nm lm</code> overrides the auto choice (LS $\rightarrow$ lm, scalar $\rightarrow$ nm); the scalar <code>-minimize</code> /Nelder-Mead path is unchanged; <code>-minimize</code> and <code>-target</code> are mutually exclusive. Parameters are still applied in place with <code>alter</code> (no re-source) and the search runs in normalized [0,1] space. All changes in <code>com_optimize.c</code> (+ one help string); no new files, no ABI change. Verified 23/23 (E-130 checks [1]–[5] + E-143 [6]–[10]): 1-param/3-target/3-stage RC magnitude fit, 2-param DC+AC multi-analysis fit ($R1=3k, R2=2k$), LM-fewer-evals-than-NM, OSDI/Verilog-A diode extraction (recover both $is \rightarrow 1e-14$ and $n \rightarrow 1.2$ from two measured I-V points by weighted least squares), and input validation (lm-without-target, minimize+target, multi-analysis+scalar all rejected)

Enhancement	Verilog files	One line
E-144	frontend/com_optimize.c, frontend/inp.c, frontend/commands.c, examples/optimize_examples/verify_optimize.py, examples/optimize_examples/optimize_dparam_demo	optimize tunes symbolic .param values via a new knob kind -dparam. -param remains an alter target (device/instance, changed in place); -dparam names a netlist .param symbol (e.g. .param w=1u, R1={500*k}) which — being expanded at parse time — is changed with alterparam <name>=<value> + a reset that re-sources the deck (re-evaluating .param expressions and re-stamping device values). Per parameter a kind (OPT_ALTER / OPT_DECKPARAM) is tracked; when any -dparam is present opt_eval applies ALL deck params first, re-sources ONCE, THEN applies the in-place alter params (reset rebuilds from the deck and would otherwise wipe an earlier alter — this ordering is what lets the two kinds mix; verified on a joint .param+device fit). No -dparam the E-130/-143 fast path is untouched (no reset, no perf hit). The two re-source banners (Reset re-loads circuit ... and Circuit: ... in inp.c) are gated by the existing ft_optimizing flag so hundreds of inner re-sources are silent (only the final leave-at-optimum run shows); ft_optimizing re-asserted after each reset. Also fixed 5 pre-existing -Wsign-conversion warnings in inp.c's *ng_script_with_params handler (argc/size unsigned). Closes the last E-143 follow-up. No new files, no ABI change. Verified 31/31 (E-130/-143 [1]–[10] + E-144 [11]–[15]): scalar .param fit (rtop→2333.3), mixed -dparam+-param joint fit (rtop=3k,R2=2k), .param in expression (k→6), least-squares -dparam (LM), quiet re-source (≤ 1 banner); AC + OSDI-device-value .param verified manually

Enhancement	mspice files	One line
E-145	frontend/com_optimize.c, frontend/commands.c, examples/optimize_examples/verify_optimize.py, examples/optimize_examples/optres.m.va, exam- ples/optimize_examples/optimize_mparam_demo	optimize tunes .model -card parameters via a third knob kind -mparam . -param remains an alter instance target and -dparam a symbolic .param (re-source); -mparam <@model[param]> names a .model -card parameter (e.g. @dmod[is] , @rmod[r]) — which is NOT alter-reachable and NOT .dc -sweepable (only sources/resistors/instance params are) — and is changed in place with altermod <name>=<value> , no re-source (as cheap per eval as -param , unlike -dparam). A new kind value OPT_MODELPARAM ; in opt_eval the deck params are re-sourced first (E-144), then the in-place knobs: alter for instances, altermod for models. @model[param] is passed straight to altermod (which accepts that accessor form), mirroring how -param @m1[w] emits alter @m1[w]=... . Circuits with only -param/-mparam keep the E-130/-143 fast path (no reset). All three knob kinds mix in one run. Change confined to com_optimize.c (+1 help line); no new source files, no inp.c change, no ABI change. Verified 39/39 (E-130/-143/-144 [1]–[15] + E-145 [16]–[20]): OSDI model-param fit (@rmod[r] → 3k), built-in diode model-param fit (@dmod[is] → 1.22e-14), determined model+instance joint fit (r=3k, R2=2k), -mparam does 0 re-sources (in-place fast path), and all three kinds (-dparam+-mparam+-param) coexist and converge

Enhancement	ngspice files	One line
E-146	frontend/com_sweep.c (new), frontend/com_sweep.h (new), frontend/commands.c, frontend/com_commands.h, frontend/inp.c, frontend/Makefile.am, examples/sweep_examples/*	universal sweep command + .sweep card. Sweeps ANY circuit knob, generalizing .dc (which steps only sources/resistors/instance params). <code>sw_kind</code> auto-detects the knob: <code>@X[y]</code> with <code>ft_sim->findModel(ckt,X)</code> non-NULL <code>?</code> model param (<code>altermod</code>), else instance (<code>alter</code>); a bare name with <code>nupa_get_param(name,&found)</code> found <code>?</code> <code>.param</code> (<code>alterparam+reset</code>), else device (<code>alter</code>) — so model params and .params , impossible with .dc , are now sweepable. <code>sweep <knob> (<start> <stop> <step> lin dec oct N a b list ...)</code> <code>[-analysis <cmd>] [-output <name>=<expr> ...]</code> : sets the knob, runs the inner analysis (default op) at each point, evaluates each output (last value; default = all node voltages like .dc), and emits a summary plot named <code>sweep</code> with the knob values as the scale (nutmeg vector API, front-end layer per E-142). The per-point analysis plots are retained (<code>tran1,tran2,...</code>). .sweep card: <code>inp.c</code> collects a top-level .sweep line into the post-parse control list as a <code>sweep</code> command; a static <code>sweep_active</code> re-entrancy guard stops a <code>.param</code> re-source (<code>reset</code> re-runs the .sweep card) from recursing. New source files registered in <code>commands.c/com_commands.h/Makefile.am</code> . Verified 11/11 (<code>verify_sweep.py</code>): <code>sweep R1</code> reproduces built-in .dc <code>R1</code> bit-for-bit, model-param (<code>@rmod[r]</code>) + <code>.param(rtop)</code> sweeps vs analytic divider, auto-detection routing, .sweep card == command form (no recursion), AC named-output vs analytic <code> H(1kHz) </code> , transient settled value, <code>lin/list/step</code> point counts, multi-output

Enhancement	ngspice files	One line
E-149	maths/misc/randnumb.c, include/ngspice/randnumb.h, frontend/numparam/xpressn.c, frontend/numparam/spicenum.c, frontend/commands.c, examples/lhs_examples/*	Latin-Hypercube Monte Carlo sampling (mcsample), closing the low-discrepancy-sampling gap vs commercial tools. A stratified sampler in randnumb.c holds the mode/N/sample-index/dimension-counter/seed; each random dimension's stratum permutation (Fisher-Yates over $0..N-1$) + per-sample jitter are generated lazily from a self-contained splitmix64 seeded by (seed, dimension) — reproducible and order-independent. mc_sample_uniform() = (perm[d][i]+jitter[d][i])/N; mc_sample_gauss() maps it through inv_normal_cdf (Acklam probit, rel err < 1.15e-9) so Gaussian tails stratify too. Sample boundary: spicenum.c's nupa_signal(NUPADECKCOPY) edge (once per reset re-source, guarded by the existing firstsignals latch) calls mc_sample_advance() (step sample, rewind dimension) before that pass's .param draws. Draw hook (xpressn.c): agauss/gauss take one mc_sample_gauss() and aunif/unif/limit one $2 \cdot \text{mc_sample_uniform}() - 1$ when LHS is active, else the unchanged gauss1()/drand() (LHS-off is byte-identical to before). Command com_mcsample (randnumb.c, registered in commands.c both tables) parses lhs <N> [seed <s>] / random / off. Front-end only, so solver-independent. Verified 5/5 under both solvers (verify_lhs.py): stratification (each of 48 strata hit once vs random's ~32/48), two-param multi-dimension, ~130x lower Var(sample-mean) at N=32 (both converge to the same mean), seed reproducibility, analytic mean/sigma correctness

Enhancement	ngspice files	One line
E-150	frontend/com_sweep.c, frontend/commands.c, maths/misc/randnumb.c, include/ngspice/randnumb.h, examples/_setup.py, examples/highsigma_examples/*	high-sigma rare-event estimation (highsigma), the last statistical ROW versus a commercial simulator -- 4-6 sigma failure probabilities (SRAM / standard-cell yield) that plain MC cannot reach (1e-7..1e-9 needs 1e7..1e9 runs). Scaled-sigma importance sampling: a new MC_MODE_SSS in the E-149 sampler draws each Gaussian .param from the lambda-inflated normal ($z = \text{lambda} * \text{gauss1}()$) and accumulates that draw's log likelihood-ratio $\log(\text{lambda}) - (z^2/2)(1 - 1/\text{lambda}^2)$ into sss_logw; $\text{mc_sample_weight}() = \exp(\text{sss_logw})$ reweights the sample so the estimator is unbiased for the nominal probability. Uniform .params are bounded so SSS leaves them un-inflated (weight 1). Direction-free -- no gradient / sensitivity / most-probable-failure-point. <code>mc_sss_config(N, lambda, seed)</code> seeds the global PRNG for reproducibility; <code>mc_sss_off()</code> reverts. Command <code>com_highsigma</code> lives in <code>com_sweep.c</code> (reuses its <code>sw_run_cmd</code> reset/analysis runner and <code>sw_eval_expr</code>), registered in <code>commands.c</code>: <code>highsigma <N> [-scale <lambda>] [-seed <s>] [-analysis <cmd>] [-metric <expr> [-max <hi>] [-min <lo>]</code> loops N samples (reset -> analysis -> eval metric -> spec test -> read weight), reports P(fail), relative error, equivalent one-sided sigma-to-fail ($-\Phi^{-1}(P)$) and failure count, and leaves them in the <code>highsigma_*</code> vectors/vars. The spec is <code>-max/-min</code> values (not a <code>>/<</code> inside the expr, which a control command reads as an I/O redirect). Front-end only, solver-independent; the verify is heavy (thousands of re-sources) so <code>examples/_setup.py</code> marks highsigma <code>SPARSE_ONLY</code>. Verified vs analytic $\Phi(-\beta)$ (<code>verify_highsigma.py</code>): $\beta=4 \rightarrow$ sigma 4.000, P 3.16e-5 (true 3.17e-5); deep tail $\beta=5$ (P 2.87e-7) recovered from 6000 runs where plain MC expects ~0.0017 failures; two-sided spec ~doubles P; seed reproducibility exact; two independent Gaussians combine as $N(., \sqrt{s1^2 + s2^2})$

Enhancement	ngspice files	One line
E-151	maths/misc/randnumb.c, include/ngspice/randnumb.h, frontend/numparam/xpressn.c, frontend/com_sweep.c, frontend/commands.c, examples/_setup.py, examples/yield_examples/*	process/mismatch correlations + packaged Monte Carlo yield -- closes the last two partial (warn) statistical rows vs a commercial simulator. (1) CORRELATIONS: <code>mc_corr_config(k, matrix)</code> Cholesky-factors a $k \times k$ correlation matrix (row-major, unit diagonal; rejects a non-positive-definite one) into lower-triangular <code>corr_L</code> ; <code>mc_corr_component(i)</code> lazily draws the k underlying z 's once per sample THROUGH <code>mc_sample_gauss()</code> (so LHS stratification / SSS weighting apply), computes $y = L*z$, caches it, and returns $y[i]$. The cache is reset in <code>mc_sample_advance()</code> which now runs its reset in EVERY mode (so correlation works under plain MC, not just LHS/SSS). New <code>.param</code> function <code>mvnorm(i)</code> (frontend/numparam/xpressn.c: added to <code>fmathS</code> , <code>XFU_MVNORM</code> enum in <code>fmathS</code> order, and the dispatch) returns the i -th correlated standard normal; with no matrix registered it degrades to an independent draw. <code>com_mccorr(mccorr <k> <m11>..<mkk></code> KLU matrix reordering + scaling controls. The KLU direct solver ran on the compiled-in <code>klu_defaults</code> (AMD ordering, max row scaling, BTF on) with no way to change them, and its one exposed knob <code>klu_memgrow_factor</code> was broken (<code>task->TSKkluMemGrowFactor = (val->rValue == 1.2)</code> stored a boolean, not the value). E-152 exposes three new <code>.options</code> -- <code>'klu_ordering=amd</code> Levenberg-Marquardt trust-region Newton (<code>.option trustregion</code> , off by default), completing the damped/trust-region-Newton row alongside the E-111 line search. In the Newton loop (<code>niiter.c</code>): the E-111 KCL-residual merit is reused (gated on <code>linesearch</code>
E-152	include/ngspice/optdefs.h, include/ngspice/tskdefs.h, include/ngspice/cktdefs.h, include/ngspice/smpdefs.h, spicelib/analysis/cktsopt.c, spicelib/analysis/cktntask.c, spicelib/analysis/cktdojob.c, maths/ni/niinit.c, maths/KLU/klusmp.c, examples/klu_tuning_examples/*	
E-153	maths/ni/niiter.c, maths/KLU/klusmp.c, include/ngspice/smpdefs.h, include/ngspice/optdefs.h, include/ngspice/tskdefs.h, include/ngspice/cktdefs.h, spicelib/analysis/cktsopt.c, spicelib/analysis/cktntask.c, spicelib/analysis/cktdojob.c, examples/trustregion_examples/*	

| E-154 | frontend/com_envelope.c (new), frontend/com_envelope.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, spicelib/analysis/envelope.c (new), spicelib/analysis/Makefile.am, include/ngspice/cktdefs.h | Envelope Following (`envelope <node> <fc> <tstop> [nppp] [m] [maxm] [reltol] [set` the last missing analysis in the RF/periodic-steady-state suite. For a carrier-driven circuit whose amplitude/phase envelope varies slowly over many carrier periods, it samples the state once per carrier period $T=1/f_c$ and integrates the slow drift, jumping M periods at a time. The exact per-period map

is $X_{n+1} = \text{phi}(X_n)$, phi = one-period DAE integration; the envelope obeys $dX/dn \sim \text{phi}(X) - X$. The naive forward-Euler jump $X_{n+M} = X_n + M(\text{phi}(X_n) - X_n)$ is UNSTABLE on high-Q circuits (the one-period monodromy has unit-circle eigenvalues, so $I + M(\text{Phi} - I)$ amplifies $\rightarrow$ blow-up; this is what shelved an earlier attempt). The new engine (spicelib/analysis/envelope.c, EFanalysis) uses the IMPLICIT backward-Euler jump $X_{n+M} = X_n + M(\text{phi}(X_{n+M}) - X_{n+M})$, solved by Newton $[(1+M)I - M*\text{Phi}] dY = -G$ with $\text{Phi} = d\text{phi}/dY$ the one-period monodromy (finite-differenced, one extra period-integration per state, dense NxN solve capped for modest circuits). A-stable, so it tracks a resonator's envelope without diverging; its fixed point $\text{phi}(X) = X$ is the true steady state. Step size M is chosen by step-doubling local-truncation-error control. The one-period map reuses the transient primitives (NicomCof + Nlitter per fixed sub-step, the dctrans state rotation) on a fixed grid of nppp points in TRAPEZOIDAL mode (backward-Euler damps a high-Q resonance); phi is SELF-STARTING (a frozen-history variant plateaued at a false low fixed point) with a sub-divided backward-Euler first sub-step to bound the restart damping. The com_envelope.c front-end command runs a short settling transient, calls EFanalysis, and emits an envelope plot ($\text{_amp} = 2|V_1|$, _dc , _re , _im vs time; nutmeg vector API). Solver-independent. Verified against a full .tran under both linear solvers: EF amplitude tracks the transient across a Q~3160 tank ring-up to <3% and to the steady state (<0.5%), stays bounded over 3000 carrier periods (26 envelope samples), and tracks a Q~316 tank to ~1.6%. Completes the RF suite |

| E-155 | frontend/com_reduce.c (new), frontend/com_reduce.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, spicelib/analysis/rcreduce.c (new), spicelib/analysis/Makefile.am, include/ngspice/cktdefs.h, examples/reduce_examples/* | RC network reduction (reduce <fmax> [factor f] [file fname] [name subckt] [keep node ...]), moving the post-layout "RC reduction / model-order reduction" gap to done. Post-layout extraction produces enormous linear parasitic R/C networks (thousands-millions of interior nodes); reduce collapses the R/C network to a small ELECTRICALLY EQUIVALENT one preserving the port behaviour over DC..fmax, written as an ordinary .subckt of R's and C's. Method (spicelib/analysis/rcreduce.c, CKTreduceRC): TICER (Time-Constant Equilibration Reduction) -- Schur-complement (Gaussian) elimination of interior nodes of $Y(s) = G + sC$, kept first order in s so the reduced network is REALIZABLE as R's and C's (no model-order-reduction black box, no passive-synthesis step). Per elimination of node n , for each neighbour pair (a,b) : $G[a,b] := G[a,n]G[n,b]/G[n,n]$; $C[a,b] := (G[a,n]C[n,b] + C[a,n]G[n,b])/G[n,n] - G[a,n]G[n,b]C[n,n]/G[n,n]^2$. The conductance update is the exact Schur complement so DC is preserved EXACTLY; the capacitance is matched to first order. A node is eliminated only when its self time-constant frequency $f_n = G_n/(2\pi C_n) > \text{factor} * f_{\text{max}}$ (quasi-static in the band of interest); factor trades reduction against in-band accuracy, monotonically. PORTS (kept nodes) are auto-detected as every node touched by a device that is NOT a resistor/capacitor (sources, transistors, OSDI Verilog-A devices), plus ground and user keep nodes, via the generic GENnode/terminal-count device interface -- so built-in and OSDI devices alike. The com_reduce.c front-end command enumerates the built-in resistor/capacitor instances (CKTtypelook), builds dense G/C over the RC nodes, runs TICER, and emits the reduced subckt (CKTnodName for node names). Solver-independent (reads devices + own dense solve). First cut is dense (node cap ~2500); sparse TICER is the follow-up. Verified under both linear solvers: a huge factor eliminates nothing and the emitted subckt reproduces the full AC response BIT-for-BIT; a moderate factor cuts nodes (e.g. 25 $\rightarrow$ 6, 4x) with <0.25 dB in-band error and DC exact; the accuracy/reduction tradeoff is monotone in factor; an OSDI device on a node auto-marks it a kept port |

| E-156 | spicelib/analysis/rcreduce.c, frontend/com_reduce.c, include/ngspice/cktdefs.h, examples/reduce_examples/* | Sparse RC reduction -- makes the E-155 reduce command scale to real post-layout size. The dense NN TICER engine (capped ~2500 nodes) is replaced by a SPARSE one: the network is stored as per-node adjacency lists (RCadj: a growable edge list of {neighbour, g, c}), and interior nodes are eliminated in a MINIMUM-DEGREE order (a lazy binary heap keyed on current degree, like sparse LU) so fill-in stays tiny -- a degree-2 chain node merges two series elements with ZERO fill. A FILL GUARD (maxdeg, default 12) declines to eliminate a node once its degree grows past the threshold, so a dense mesh core (whose boundary Schur complement is inherently dense) is left intact instead of densifying

the whole network (measured: unguarded, one mesh elimination created 3308 fill edges; guarded, 9). The TICER Schur-complement update, DC-exact conductance, frequency criterion (eliminate only when $f_n = G_n / (2\pi C_n) > \text{factorfmax}$), and automatic port detection are unchanged from E-155; in the edge representation the diagonal is implicit so eliminating a node just adds Schur edges among its neighbours. Node cap raised 2500 -> 5,000,000; a 65,017-node network reduces in ~4 s. New maxdeg command option (com_reduce.c) and CKTreduceRC parameter (cktdefs.h). Also fixes a terminal-ordering pitfall: the reduced .subckt's terminals are emitted in internal-index order (not the order keep nodes were typed), so instantiating with the wrong order silently swaps ports -- the command now prints the correct x1 <ports...> <name> instantiation line. Verified under both linear solvers: identity reduction bit-exact (0.00 dB), a moderate factor gives ~4x fewer nodes at <0.25 dB in-band with DC exact, monotone accuracy/reduction tradeoff, OSDI auto-port, and an 8001-node network (past the old dense cap) reduces successfully | | E-157 | frontend/com_aging.c (new), frontend/com_aging.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, examples/aging_examples/* | Device aging (aging <t_target> [rate <opvar>] [param <ageparam>] [dynamic <tstop> [tstep]] [verbose]), adding the reliability "stress -> degrade -> re-simulate (fresh/aged)" flow (HCI/NBTI/TDDDB) and moving those reliability gap rows to done. The command finds every aging-capable device in the loaded circuit, computes how much it has degraded after t_target seconds of operation, writes that back into the device, and RE-STAMPS the circuit so any analysis run afterwards (op/dc/tran/ac) sees the aged devices; it runs the fresh stress simulation first, leaving it as the current plot (a "fresh" baseline). The design is MODEL-AGNOSTIC: a device opts in purely by exposing, in its Verilog-A/OSDI model, a degradation-RATE operating-point variable (default agerate, in dose-units/second) and a per-instance AGE parameter (default age, (*type="instance"*)). The engine's only job is to integrate the rate into a dose and feed it back; the model owns the physics that maps age to a parameter shift (a sublinear power law, an Arrhenius factor, ...). Participants are detected by scanning each device TYPE's instance-parameter table (DEVices[t]->DEVpublic.instanceParms) for the two keywords, so ordinary resistors/sources are silently skipped and probing never errors. Two modes: STATIC (default) reads the rate at the DC operating point and scales by the lifetime, age = agerate(op)t_target; DYNAMIC (dynamic <tstop>) runs a transient over one representative window, trapezoidally (time-weighted) integrates the rate, and extrapolates, age = (INT agerate dt / tstop)t_target -- capturing duty cycle. Because age is per-instance, devices sharing a .model but at different bias age independently. Implementation (frontend/com_aging.c) reuses the com_optimize synchronous command-dispatch pattern to run op/tran, the nutmeg expression engine to read @inst[agerate], and alter @inst[age]=... to write back; console chatter suppressed via ft_optimizing unless verbose. Solver-independent. Verified under both linear solvers with a demo square-law NMOS carrying an NBTI hook (agemos.va): enumeration picks exactly the ageable devices, the dose is exactly ratet_target, the threshold shift matches the analytic $\Delta V_{th} = dv_{th_ref}(\text{age}/\text{age_ref})^{0.25}$ law to 5 sig figs, aged drain current drops, degradation is monotone in lifetime, a near-threshold device loses a larger fraction of its current than a hard-driven one for the same shift, a 30%-duty pulsed gate ages at 0.30x the DC rate (dynamic), and a sub-threshold device accrues zero dose | | E-158 | frontend/com_emir.c (new), frontend/com_emir.h (new), frontend/commands.c, frontend/com_commands.h, frontend/Makefile.am, examples/emir_examples/* | Power-grid EMIR (emir [rail V] [thresh frac] [thick m] [jmax A/m2] [n exp] [tref s] [top k] [verbose]), adding electromigration + IR-drop reliability sign-off and moving the last reliability gap row to done. After a DC solve of the power-distribution network the command reports two metrics. IR-DROP: for every node, the drop below the ideal rail (rail - V(node)); the rail defaults to the highest node voltage (the supply pad) unless given; reports the worst node and every node past a threshold (default 10% of the rail). ELECTROMIGRATION: for every wire-segment resistor, the current DENSITY $J = |I|/(w\text{thickness})$, ranked; the worst-density segment, every segment past the current-density limit Jmax, and a Black's-equation relative lifetime MTTF/ref = (Jmax/J)^n (Black: MTTF ~ J^-n; a segment exactly at Jmax has MTTF = the reference lifetime tref, needing no process-specific prefactor). The physics the analysis exists to expose: EM is set by current DENSITY, not current -- a fat trunk carrying huge current can be safe while a thin wire at a fraction of that current voids first, which is why real grids taper wire width with carried current to hold J roughly constant. Implementation (frontend/com_emir.c) runs a fresh op (com_optimize synchronous-dispatch

pattern), walks the current plot's node-voltage vectors (`plot_cur->pl_dvecs` filtered to `SV_VOLTAGE`) for IR-drop, and enumerates the resistor instances (the device type named "Resistor") reading each segment's current and width via the nutmeg expression engine (`@Rk[i]`, `@Rk[w]`) -- so it needs no device-struct access and works for any resistor; segments without a width are skipped and counted. Both tables are qsort-ranked (IR by drop, EM by density) and truncated to top. Solver-independent (reads a DC solution + per-resistor currents). Verified under both linear solvers on a 3-segment tapered ladder off a 1 V rail: worst IR drop 0.30 V (30%) at the far tap matching the exact ladder solve and linear in load current; $J = I/(w\text{thick})$ exact; the worst-density segment is the narrow low-current wire, not the high-current wide trunk; the Black MTTF ratio between two segments equals $(J2/J1)^n$; with J_{max} between the trunk and leaf density exactly the under-sized segments fail; rail auto-detect equals an explicit rail; and an OSDI (Verilog-A) current load at a tap is handled identically to a current source | | [E-159](#) | (no ngspice source change) `examples/compactmodels_examples/*` | Real production compact-model bring-up -- a validation/example enhancement that exercises the EXISTING toolchain on actual CMC (Compact Model Coalition) Verilog-A models, so no ngspice or openvaf-r source change. BSIM4 (the ~12,600-line industry-standard bulk MOSFET) is compiled through openvaf-r to OSDI (~6 s) and validated against ngspice's BUILT-IN BSIM4 -- the same model, so a rigorous self-check: the OSDI BSIM4.8 and native BSIM4.8.3 agree to ~2% on the Id-Vds output family and ~4.5% on the Id-Vgs transfer curve (the residual is the point-release version gap). EKV (EPFL compact MOSFET, which ngspice has NO built-in for) is brought up purely through OSDI, giving textbook saturation. A general finding surfaced: OSDI keeps every internal node STATIC (no dynamic node collapsing), so BSIM4's internal drain/source nodes (`di/si`) -- which the default `rdsmod=0` expects the simulator to collapse onto the external d/s -- are left floating and the device conducts ZERO current; enabling the external series-resistance nodes with `rdsmod=1` (the model near-shorts them, 1e3 mho, when no S/D geometry is given) connects them and the device works. Models are compiled in place from the bundled OpenVAF integration-test sources (licenses stay with them). Verified under both linear solvers (6 checks) | | [E-160](#) | (no ngspice source change) `examples/cmcsweep_examples/`, `examples/_setup.py` | CMC compact-model coverage sweep -- the comprehensive follow-up to E-159, again a validation/example enhancement with no ngspice/openvaf-r source change (only `_setup.py` gains `cmcsweep` in its `SPARSE_ONLY` and `REGRESSION_EXCLUDE` sets so the slow 19-model compile runs single-solver and off the fast sweep). It drives EVERY real CMC (Compact Model Coalition) Verilog-A model bundled with OpenVAF through openvaf-r -> OSDI -> ngspice and reports a coverage matrix, the same idea as the E-84 LRM sweep but for production compact models. Result: 19 of 19 models COMPILE and 19 of 19 LOAD, spanning every device class -- MOSFET (BSIM3, BSIM4, BSIM6, BSIMBULK, EKV, HiSIM2, HiSIMSOTB, MVSG_CMC), FinFET (BSIM-CMG, BSIMIMG), SOI (BSIMSOI, HiSIMHV), GaN HEMT (ASMHEMT), surface-potential (PSP102, PSP103), bipolar/SiGe-HBT (HICUML2, MEXTRAM), and diode (DIODE, DIODE_CMC). Deeper conduction+physics validation for a representative model per class: OSDI BSIM4 and BSIM3 match ngspice's BUILT-IN BSIM4/BSIM3 to ~2%/~7%; EKV gives monotonic saturating I-V; HICUML2 shows bipolar current gain $\beta \sim 100$ on a Gummel plot; DIODE_CMC turns on exponentially ($\times 16000$ over 0.2-1.0 V). Two findings: (1) the E-159 internal-node/`rdsmod=1` issue is BSIM4-SPECIFIC, not universal -- BSIM3 handles the zero-resistance case fine and conducts out of the box (its apparent zero at $V_{\text{gs}}=1$ V is just a high ~1.7 V default threshold); (2) HiSIMHV (6 terminals `d,g,s,b,sub,temp`) needs `cosubnode=1` to enable the substrate node, and with the default `cosubnode=0` its `$fatal` correctly guards the node-count mismatch (confirming `$fatal` works through OSDI). Models compiled in place from the bundled integration-test sources (licenses stay put). Verified (7 checks); the compile/load tiers are solver-independent | | [E-161](#) | (no ngspice source change) `examples/dynmodels_examples/` | Dynamic (AC/RF) compact-model validation -- extends the E-159/160 DC validation into the dynamic domain, again a validation/example enhancement with no ngspice/openvaf-r source change. Where E-159/160 exercised DC I-V, this exercises the DYNAMIC behavior, which flows through a different code path: OSDI's REACTIVE (charge) Jacobian stamping and ngspice's `.ac` analysis, on the real production models (compiled in place from the bundled CMC sources). BSIM4 C-V: the gate capacitance $C_{\text{gg}}(V_{\text{gs}})$ is extracted from `.ac` as $\text{Im}(I_{\text{gate}})/\omega$ and swept over gate bias; it rises from ~40 fF in subthreshold (overlap+fringe) to ~129 fF in inversion (approaching $C_{\text{ox}}W/L$) -- the textbook MOSFET C-V -- and the OSDI-compiled BSIM4 matches ngspice's BUILT-IN BSIM4 to under 1% at every bias (a tighter check than the ~2% DC

I-V match, since Cgg is dominated by the version-independent oxide term). Cutoff frequency f_T (the frequency where AC current gain $|h_{21}|=|I_{out}/I_{in}|$ falls to 1): OSDI BSIM4 $f_T \sim 3.5$ GHz matches the built-in to $\sim 1\%$; HICUML2 (SiGe HBT, which ngspice has no built-in for) has zero default transit time ($t_0=0$) giving infinite f_T , so a realistic dynamic parameter set ($t_0=10$ ps, 1 fF junction caps) is supplied and the resulting f_T lands right at the transit-time limit $1/(2\pi t_0) \sim 15.9$ GHz and rises with collector current -- textbook charge-control behavior. Verified under BOTH the Sparse and KLU solvers (.ac is supported by both), 4 checks | | [E-162](#) | frontend/inp.c, examples/hb_examples/verify_hb.py | **.hb** dot-card for harmonic balance -- gives the E-134 hb command netlist dot-card parity with the PSS family (.pss/.pac/.pnoise/.pxf/.psp/.sp are dot-cards, while the HB family hb/qpss/hbosc were control-block commands only). A top-level **.hb** <f0> <K> [points] [maxiter] card now runs single-tone harmonic balance straight from the deck (no .control block needed). Rather than duplicating the HB machinery as a separate analysis JOB, **.hb** reuses the deck->control mechanism Enhancement-146 introduced for .sweep: during deck loading (frontend/inp.c) a top-level **.hb** ... line is stripped of its leading . and appended to the post-parse control list, so it executes as hb ... (the com_hb command) once the circuit is built -- inheriting all of the E-134 engine (argument parsing, the E-121 conversion-matrix Jacobian, E-135 source-stepping continuation, solver independence). A boundary check (next char after .hb is whitespace or end-of-line) keeps it from swallowing any future **.hb*** card. Before this, **.hb** produced unimplemented dot command '**.hb**' -- the only **.hb** handler in ngspice was a disabled `#ifdef WITH_HB` upstream stub (dot_hb) that merely redirected to the PSS analysis, not the E-134 engine. Verified (examples/hb_examples/verify_hb.py, +2 checks): the **.hb** dot-card run in plain batch mode (no .control) converges and produces a harmonic-balance spectrum bit-for-bit identical to the hb command form on a junction-limited diode rectifier; it threads its optional [points]/[maxiter] arguments and coexists with a following .control block (deck order preserved); both Sparse and KLU give identical results. (Like .sweep, a bare command-style dot-card with no other analysis card prints a benign "no simulations run" batch notice even though HB ran.) | | [E-163](#) | frontend/inp.c, examples/qpss_examples/verify_qpss.py, examples/phasenoise_examples/verify_phasenoise.py | **.qpss** / **.hbosc** / **.phasenoise** dot-cards -- completes the harmonic-balance family's netlist dot-card parity begun by E-162's **.hb**. Three more top-level command-style cards now run their analyses straight from the deck: **.qpss** <expr> <f1> <f2> [periods] [maxorder] (two-tone quasi-periodic steady state, E-133/136; add the hb keyword for the frequency-domain form), **.hbosc** <oscnode> <K> [fguess] [tstab] (autonomous harmonic balance for an oscillator, E-140; the deck needs a .ic to start the oscillation), and **.phasenoise** <fstart> <fstop> [points] (oscillator phase noise, E-140; run after **.hbosc**). Each reuses the same one-branch deck->control mechanism as **.hb** (E-162) / .sweep (E-146): in frontend/inp.c a top-level **.qpss**/.**hbosc**/.**phasenoise** line is stripped of its leading . and appended to the post-parse control list, executing as the corresponding command once the circuit is built -- inheriting the full E-133/136/140 engines and their solver independence. Because the bridge preserves deck order, **.hbosc** (which finds the oscillator PSS + frequency) and **.phasenoise** (which reads it) compose, so a complete oscillator phase-noise run needs no .control block. A per-card boundary check (the char after the name is whitespace or end-of-line) keeps the cards distinct -- in particular **.hbosc** is NOT matched by the **.hb** branch, whose own boundary check rejects the trailing osc. Verified: the **.qpss** dot-card produces a two-tone spectrum bit-for-bit identical to the qpss command (examples/qpss_examples/verify_qpss.py); **.hbosc** + **.phasenoise** in a deck reproduce the command form's oscillation frequency and phase-noise spectrum exactly with order preserved (examples/phasenoise_examples/verify_phasenoise.py); both under Sparse and KLU. No regression to **.hb**/.sweep. |

(E-25's simparam exposure lives in the OSDI callback table populated at load time; its diff rides inside the osdi_load.c/callbacks changes.)

Build-warning cleanup (post-E-127): a full rebuild under the repo's -Wconversion flags surfaced latent warnings. configure.ac had -s (a strip flag) in the compile CFLAGS, so clang warned "argument unused during compilation: '-s'" on every source file (~1526 warnings) — removed (binaries are stripped separately by the fold and CI). -Wconversion also flagged a real latent bug in maths/ni/nipzmeth.c (`b = 0.0` was an assignment where `b == 0.0` was intended) — fixed — and benign double→bool conversions in

maths/cmaths/cmath4.c — made explicit. Full-build warnings dropped 1903 → 394; the remainder are *-Wconversion* warnings in third-party SuiteSparse (KLU) and CMC device models (HiSIM), which are left to their upstream sources.

Every entry above is guarded by a verify script under [examples/](#) and was regression-checked against the full example arsenal, the compiler's integration suite, and the VA_TEST industry-model corpus at the time it landed.